Chaîne CI/CD de l'API SouverainRecrut

Rapport technique

Dépôt GitHub	github.com/mtbvr/CICD_souverainrecrut, branche dev
Projet GitLab	gitlab.com/mtbvr/souverainrecrut-deploiement
Image Docker	hub.docker.com/r/mtbvr/souverainrecrut-backend
API déployée	51.255.206.188:4000/v1/health, documentation sur 51.255.206.188:4000/api-docs

Sommaire

1. Objectif 1

Partie 1 : intégration continue **2**

2. Architecture du lab 2
3. Tests et couverture 2
4. Analyse SonarQube 3
5. Image Docker et Trivy 4
6. Pipeline Jenkins 4

Partie 2 : déploiement **6**

7. Serveur et Terraform 6
8. Préparation du serveur avec Ansible 6
9. GitLab : runner et variables 7
10. Déploiement avec Docker Compose 7
11. Pipeline de déploiement 8
12. Rollback 8
13. Vérifications 9

1. Objectif

Le but du projet est d'automatiser la livraison de l'API Backend, du push sur la branche dev jusqu'à l'API en ligne sur un serveur.

Ça se fait en deux temps. D'abord l'intégration continue : à chaque push sur dev, Jenkins teste et analyse le code, puis publie l'image Docker sur Docker Hub. Ensuite le déploiement : Terraform et Ansible préparent le serveur, et un pipeline GitLab récupère l'image sur Docker Hub, sans la reconstruire, pour la déployer avec Docker Compose.

Ce qui relie les deux, c'est le tag de l'image. On choisit le tag à déployer, et on reprend un ancien tag pour revenir en arrière.

Partie 1 : intégration continue

2. Architecture du lab

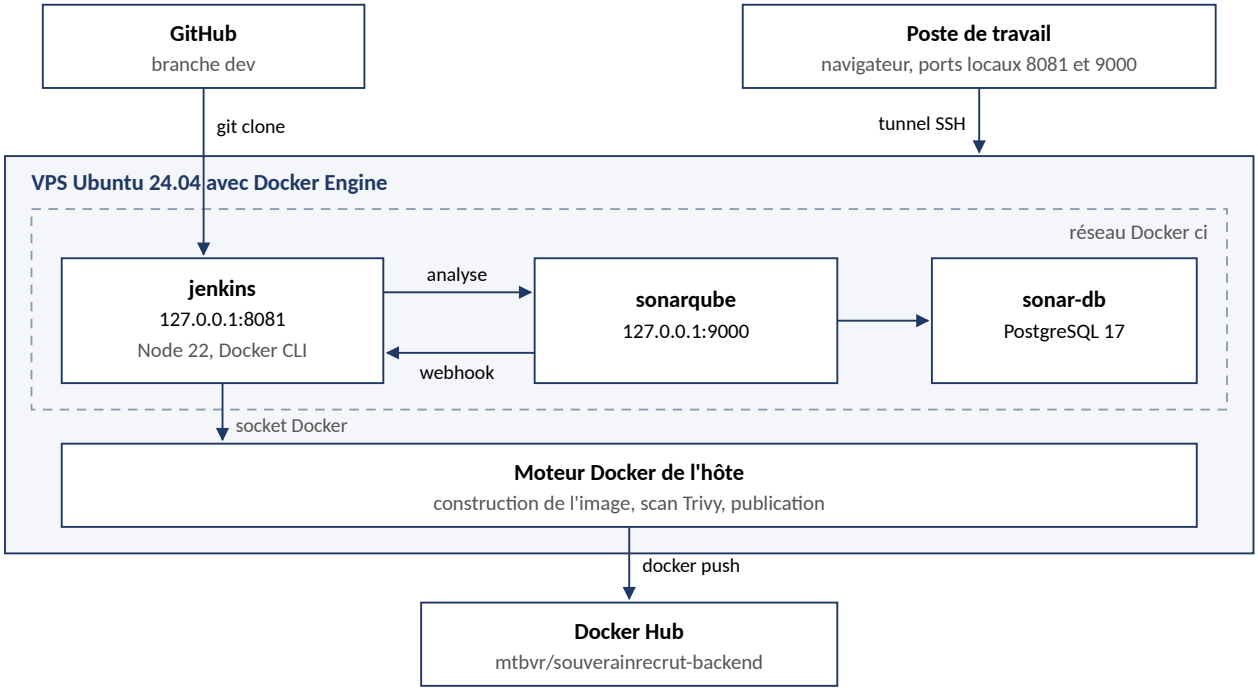


Figure 1 : architecture de l'intégration continue

Jenkins, SonarQube et sa base PostgreSQL tournent en Docker Compose sur mon VPS. Les interfaces ne sont pas exposées sur Internet, j'y accède depuis mon PC avec un tunnel SSH.

L'installation du lab est scriptée, et le script check-lab.sh vérifie que tout tourne (figure 2).

```
ubuntu@vps-841b4090:~/CICD_souverainrecrut/integration-continue/lab$ ./scripts/check-lab.sh
== Conteneurs
NAME        STATE     STATUS
jenkins     running   Up 3 minutes
sonar-db    running   Up 4 hours (healthy)
sonarqube   running   Up 4 hours

== Services
  [OK]   Jenkins répond (port 8081)           HTTP 200
  [OK]   SonarQube status UP (port 9000)       "status":"UP"
  [OK]   Jenkins -> SonarQube (réseau ci)      HTTP 200

== Outils dans le conteneur Jenkins
  [OK]   Node.js >= 22                         v22.23.4
  [OK]   npm                                   10.9.9
  [OK]   Docker CLI -> daemon de la VM         Server 29.9.2
  [OK]   Docker buildx                         github.com/docker/buildx v0.38.1 0dbe87f36ed472fe36b6796c3f70b4210a731fc
5
  [OK]   git                                   git version 2.47.3

Lab opérationnel.
```

Figure 2 : vérification du lab avec check-lab.sh

3. Tests et couverture

Le dépôt de départ n'avait aucun test. J'en ai écrit 97 avec Jest et Supertest, sur l'authentification, les candidats, les offres, les candidatures, les utilisateurs et le contrôle d'accès par rôle. Le client Prisma est mocké, du coup pas besoin de base de données pour lancer les tests.

La couverture est à 100 %, avec un seuil à 80 % qui fait échouer le build en dessous (figure 3).

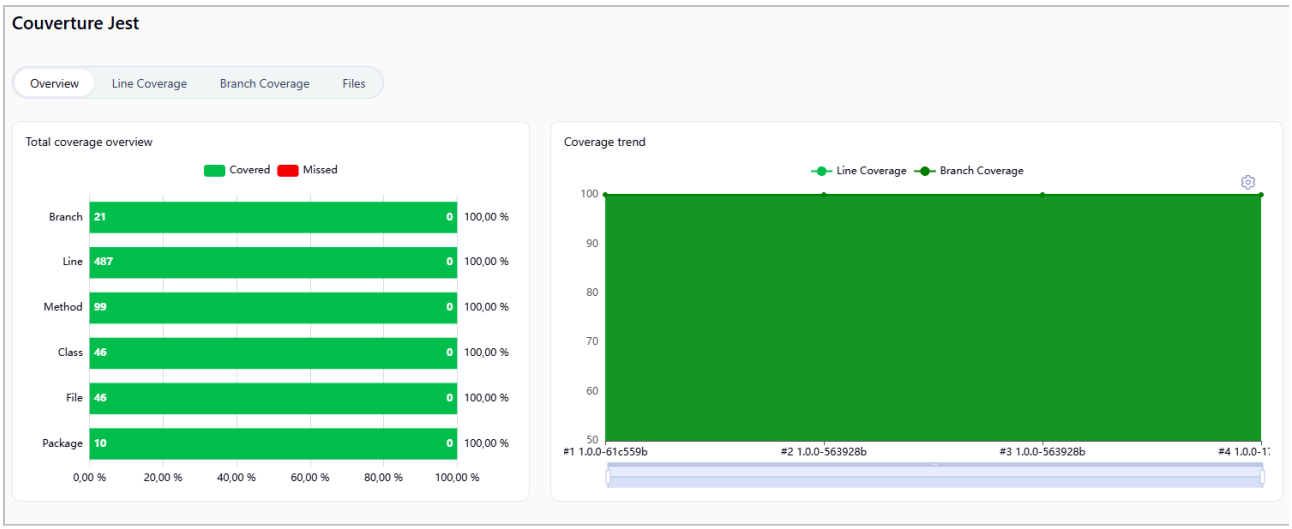


Figure 3 : couverture de code dans Jenkins

4. Analyse SonarQube

Le Quality Gate par défaut (Sonar way) ne regarde que le nouveau code. À la première analyse il laissait tout passer, alors j'ai créé un Quality Gate SouverainRecrut qui porte sur tout le code : au moins 80 % de couverture, moins de 3 % de duplication, la note A partout et tous les Security Hotspots revus.

Le build 3 s'est fait bloquer par ce Quality Gate (figure 4). Il fallait retirer l'en-tête X-Powered-By, qui indiquait qu'on utilise Express, et remplacer deux parseInt par Number.parseInt dans la pagination. Au build 4, tout est en A (figure 5).

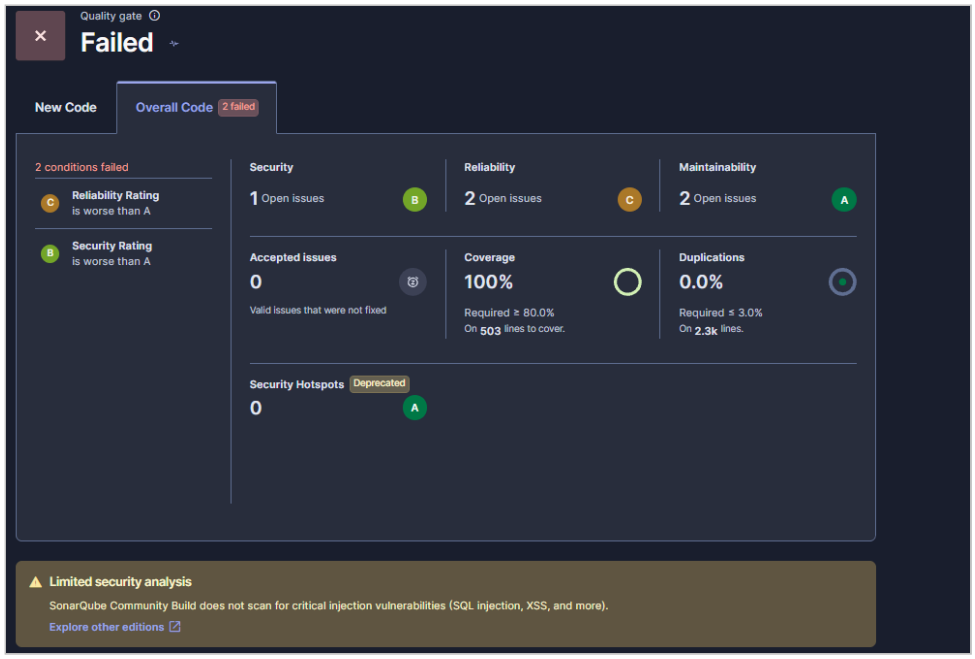


Figure 4 : Quality Gate bloqué au build 3

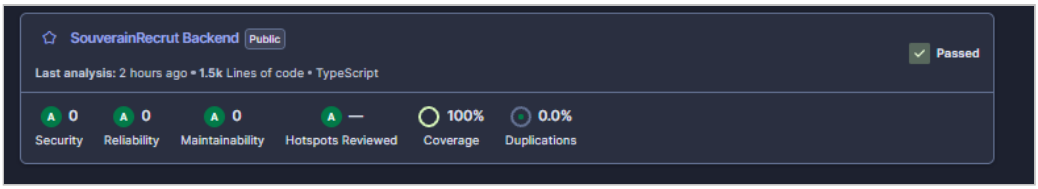


Figure 5 : Quality Gate validé au build 4

5. Image Docker et Trivy

J'ai repris le Dockerfile fourni : Node 22 Alpine, build en deux étapes pour ne garder que le code compilé et les dépendances de prod, utilisateur non root, healthcheck et .dockerignore. Chaque image est taguée version-commit, par exemple 1.0.0-8b039e7, ce qui permet de retrouver le code exact d'une image.

Pour Trivy, le build est bloqué seulement sur les failles critiques qui ont un correctif, le reste est noté dans le rapport. Au build 1, il a trouvé une faille critique dans proxy-addr, une dépendance d'Express (figure 6). Une mise à jour du package-lock.json a suffi pour le build 2.

```
14:35:35  Node.js (node-pkg)
14:35:35  ==================
14:35:35  Total: 1 (CRITICAL: 1)
14:35:35
14:35:35  ┌──────────────────────┬────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────┐
14:35:35  │       Library        │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                Title                 │
14:35:35  ├──────────────────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────┤
14:35:35  │ proxy-addr (package.json) │ CVE-2026-90711 │ CRITICAL │ fixed  │ 2.0.7             │ 2.0.8         │ proxy-addr: proxy-addr vulnerable to IP spoofing via │
14:35:35  │                      │                │          │        │                   │               │ IPv4-mapped IPv6 trust subnet        │
14:35:35  │                      │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-90711 │
14:35:35  └──────────────────────┴────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────┘
```

Figure 6 : faille critique trouvée par Trivy au build 1

J'ai ensuite enlevé npm de l'image finale et mis à jour quelques dépendances pour réduire les failles HIGH. Cette version est sortie au build 5 :

Image	CRITICAL	HIGH	MEDIUM	LOW
1.0.0-172892d (build 4)	0	26	20	3
1.0.0-8b039e7 (build 5)	0	1	4	0

La dernière HIGH vient d'une dépendance de Prisma qui ne sert qu'aux migrations, elle partira avec Prisma 7.

6. Pipeline Jenkins

Le pipeline est dans integration-continue/Jenkinsfile. Les étapes s'enchaînent dans cet ordre : checkout, npm ci, génération Prisma, compilation TypeScript, tests et couverture, analyse SonarQube, Quality Gate, build de l'image, scan Trivy et publication sur Docker Hub.

Dès qu'une étape échoue, le pipeline s'arrête. La publication étant la dernière étape, une image n'arrive sur Docker Hub que si tous les contrôles sont passés. Les tokens SonarQube et Docker Hub sont stockés dans les credentials Jenkins, pas dans le dépôt.

Sur les cinq builds, deux ont été bloqués :

Build	Image	Résultat
1	1.0.0-61c559b	Bloqué par Trivy
2	1.0.0-563928b	Publié
3	1.0.0-563928b	Bloqué au Quality Gate
4	1.0.0-172892d	Publié
5	1.0.0-8b039e7	Publié, version de référence



Figure 7 : étapes des builds 1 à 4 dans Jenkins

```
15:52:26  Image publiée : <compte>/souverainrecrut-backend:1.0.0-172892d (+ tag dev)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
15:52:26  Archiving artifacts
[Pipeline] sh
15:52:26  + docker image ls --format {{.Repository}}:{{.Tag}}
```

Figure 8 : publication de l'image sur Docker Hub en fin de build

Partie 2 : déploiement

7. Serveur et Terraform

Le déploiement se fait sur le même VPS. Terraform et Ansible sont lancés depuis mon PC, dans WSL, et se connectent au serveur en SSH avec un utilisateur deploy et sa propre clé.

Comme le VPS existe déjà, Terraform n'a pas de machine à créer. Il vérifie les prérequis du serveur (figure 9), génère l'inventaire Ansible et sort les valeurs à mettre dans GitLab (figure 10).

```
terraform_data.vm_check: Creating...
terraform_data.vm_check: Provisioning with 'remote-exec'...
terraform_data.vm_check (remote-exec): Connecting to remote host via SSH...
terraform_data.vm_check (remote-exec):   Host: 51.255.206.188
terraform_data.vm_check (remote-exec):   User: deploy
terraform_data.vm_check (remote-exec):   Password: false
terraform_data.vm_check (remote-exec):   Private key: true
terraform_data.vm_check (remote-exec):   Certificate: false
terraform_data.vm_check (remote-exec):   SSH Agent: false
terraform_data.vm_check (remote-exec):   Checking Host Key: false
terraform_data.vm_check (remote-exec):   Target Platform: unix
local_file.environment_summary: Creating...
local_file.environment_summary: Creation complete after 0s [id=ea866a56e000401d344a8fffa364e9bdddab2f2e]
terraform_data.vm_check (remote-exec): Connected!
terraform_data.vm_check (remote-exec): Hôte  : vps-841b4090
terraform_data.vm_check (remote-exec): OS    : Ubuntu 24.04.5 LTS
terraform_data.vm_check (remote-exec): vCPU  : 6
terraform_data.vm_check (remote-exec): RAM   : 11676 Mo
terraform_data.vm_check (remote-exec): Disque: 96G
terraform_data.vm_check (remote-exec): python3 : OK
terraform_data.vm_check (remote-exec): sudo sans mot de passe : OK
terraform_data.vm_check (remote-exec): Accès Internet : OK
terraform_data.vm_check: Creation complete after 3s [id=c1a93e7d-dcc9-597b-f2c7-42db147764ae]
local_file.ansible_inventory: Creating...
local_file.ansible_inventory: Creation complete after 0s [id=be05bf636c26cca040c0d5f1488dd42fbe41d981]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

Figure 9 : terraform apply, vérification du serveur

```
Outputs:

ansible_command = "cd ../ansible && ansible-playbook -i inventory.ini playbook.yml"
ansible_inventory = "/home/mateo/CICD_souverainrecrut/deploiement/ansible/inventory.ini"
api_url = "http://51.255.206.188:4000/v1/health"
gitlab_variables = {
  "API_PORT" = 4000
  "DEPLOY_DIR" = "/opt/souverainrecrut"
  "DEPLOY_HOST" = "51.255.206.188"
  "DEPLOY_USER" = "deploy"
}
ssh_command = "ssh -i ~/.ssh/souverainrecrut_deploy deploy@51.255.206.188"
```

Figure 10 : valeurs sorties par Terraform

8. Préparation du serveur avec Ansible

Le playbook met à jour le système, installe Docker et Docker Compose, ajoute deploy au groupe docker, crée le répertoire /opt/souverainrecrut et ouvre le port de l'API dans le pare-feu. Il finit par un conteneur de test pour vérifier que Docker marche (figure 11).

```
TASK [Afficher le résultat] ***********************************
ok: [souverainrecrut-vm] => {
    "msg": [
        "Docker Engine : 29.9.3",
        "Docker Compose : 5.7.0",
        "Conteneur de test : OK",
        "Répertoire de déploiement : /opt/souverainrecrut"
    ]
}
```

Figure 11 : vérifications en fin de playbook

Au deuxième lancement, rien n'a changé (changed=0), on peut donc le rejouer sans risque (figure 12).

```
PLAY RECAP *********************************************************************************************************
souverainrecrut-vm         : ok=17   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ign
ored=0
```

Figure 12 : deuxième lancement du playbook

9. GitLab : runner et variables

Le déploiement passe par un projet GitLab à part, souverainrecrut-deploiement. Ses jobs tournent sur un runner installé par Ansible sur le VPS, dans un conteneur Docker. Les jobs demandent le tag de ce runner (souverainrecrut-deploy), ils passent tous par lui (figure 13).

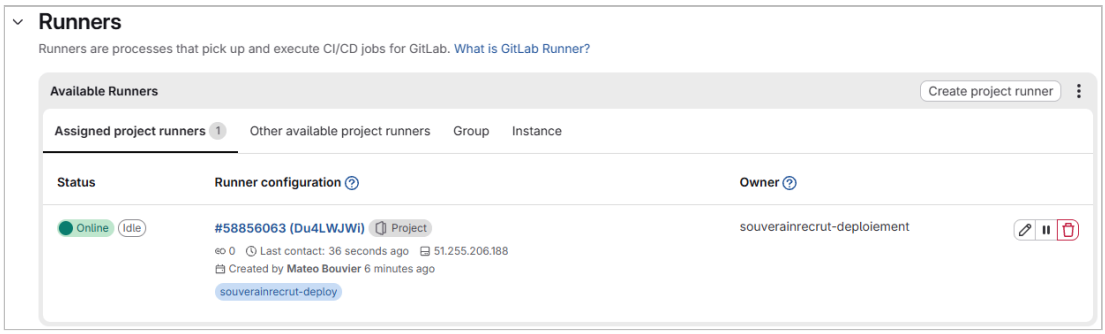


Figure 13 : runner du projet en ligne sur GitLab

Les secrets sont dans les variables CI/CD du projet : token Docker Hub en lecture seule, clé SSH, mot de passe PostgreSQL et secret JWT (figure 14). Ils sont masqués et protégés, ils n'apparaissent pas dans les logs et ne sont envoyés qu'aux pipelines de main.

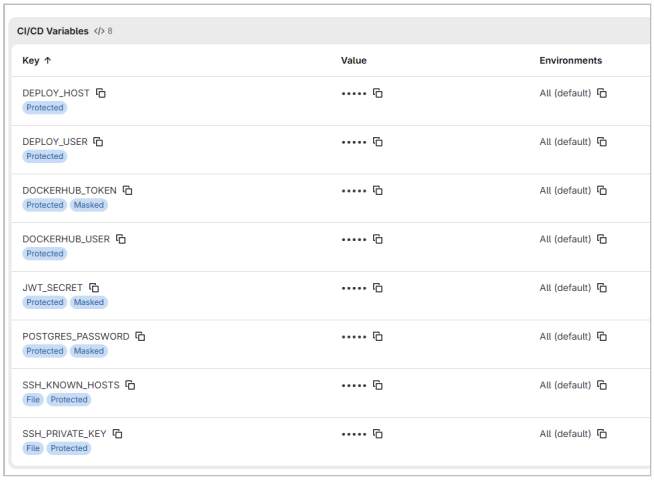


Figure 14 : variables CI/CD du projet GitLab

10. Déploiement avec Docker Compose

Le docker-compose.yml lance trois services : PostgreSQL, un conteneur migrate qui applique les migrations Prisma, puis l'API. L'API ne démarre que si la base est prête et que les migrations ont réussi. Le conteneur migrate utilise la même image que l'API, les migrations sont toujours celles de la version déployée.

Les données sont dans un volume Docker et la base n'expose aucun port. L'API et la base ont un healthcheck et redémarrent toutes seules si elles s'arrêtent.

11. Pipeline de déploiement

On lance le pipeline à la main dans GitLab, avec le tag de l'image à déployer (figure 15). Il vérifie que le tag est valide et que l'image existe sur Docker Hub, valide le docker-compose.yml, puis attend une approbation manuelle avant de déployer (figure 16).

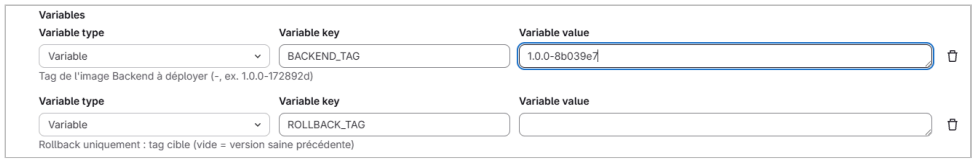


Figure 15 : lancement du pipeline avec le tag à déployer

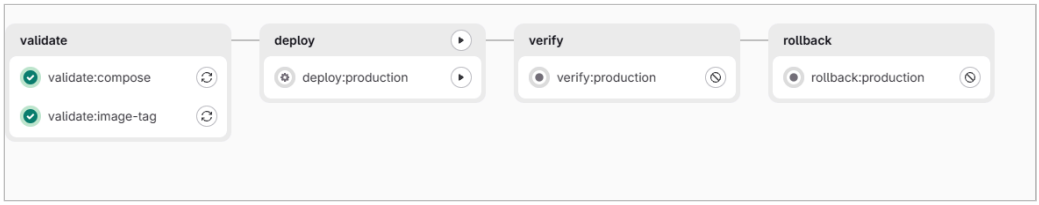


Figure 16 : déploiement en attente d'approbation

Une fois approuvé, le job se connecte au serveur en SSH, envoie le docker-compose.yml et les scripts, génère le .env à partir des variables GitLab et lance deploy.sh. Ce script récupère l'image, démarre la stack, attend que l'API soit healthy et lance les tests de fumée. Si ça échoue, il remet tout seul la version d'avant.

Le job verify contrôle ensuite que c'est bien le bon tag qui tourne et repasse les tests de fumée (figures 17 et 18). J'ai d'abord déployé 1.0.0-172892d, puis 1.0.0-8b039e7.

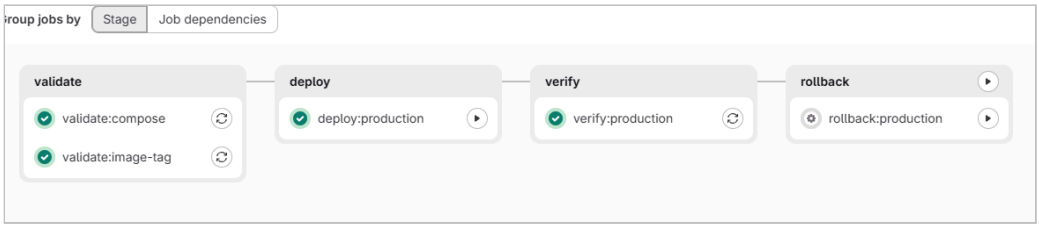


Figure 17 : pipeline de déploiement de 1.0.0-8b039e7

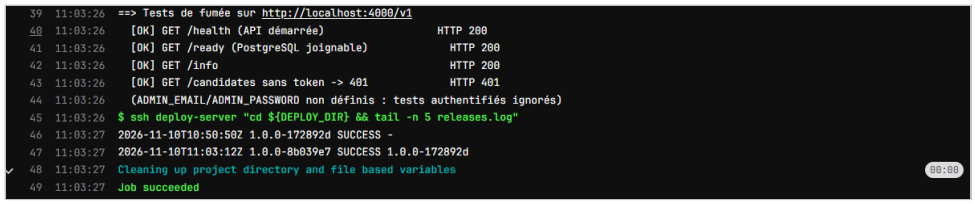
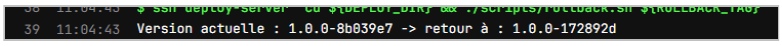


Figure 18 : tests de fumée et historique dans le job verify

12. Rollback

Le rollback est un job manuel du pipeline. Sans tag précisé, il revient à la dernière version saine notée dans releases.log, l'historique tenu par le script de déploiement.

Je l'ai testé après le déploiement de 1.0.0-8b039e7 : retour sur 1.0.0-172892d, avec les mêmes contrôles qu'un déploiement (figure 19). J'ai ensuite redéployé 1.0.0-8b039e7 (figure 20).



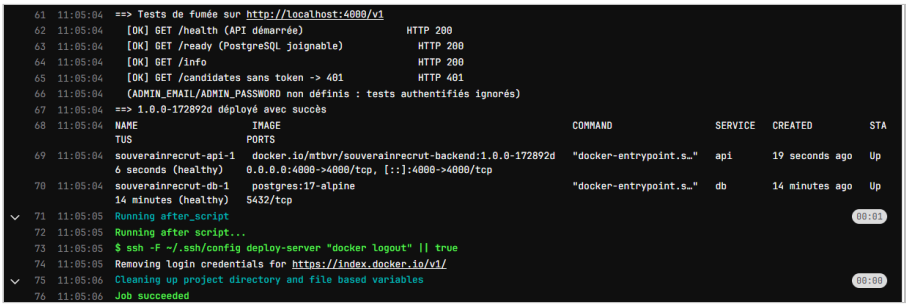


Figure 19 : rollback de 1.0.0-8b039e7 vers 1.0.0-172892d

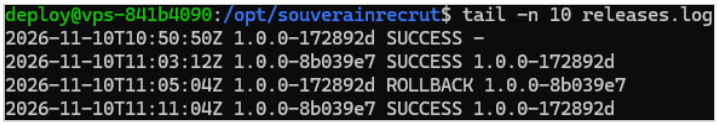


Figure 20 : historique des déploiements dans releases.log

Le rollback ne défait pas les migrations Prisma. Ici aucun souci, les deux versions ont le même schéma, mais pour une vraie mise à jour il faut des migrations qui restent compatibles avec la version d'avant.

13. Vérifications

Au final, l'API tourne sur 1.0.0-8b039e7 et les deux conteneurs sont healthy (figure 21).

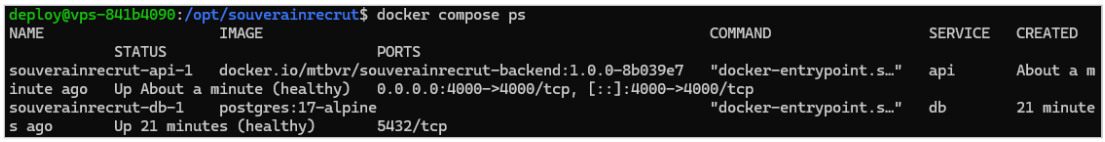


Figure 21 : état des conteneurs sur le serveur

Comme l'API n'a pas de route d'inscription, j'ai créé le compte admin directement sur le serveur avec create-admin.sh. La doc Swagger du projet pointe sur localhost:4500, je l'ai ouverte via un tunnel SSH vers le port 4000 du serveur. Avec le compte admin, j'ai créé un candidat (figure 22).

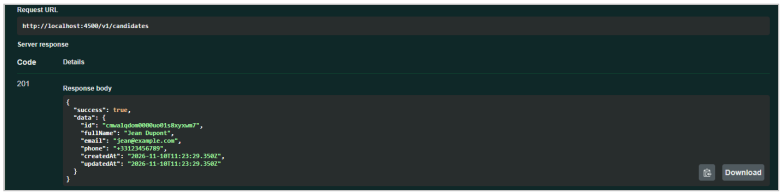


Figure 22 : création d'un candidat depuis Swagger

Après un docker compose down puis up, le candidat est toujours là, les données sont bien gardées dans le volume (figure 23).

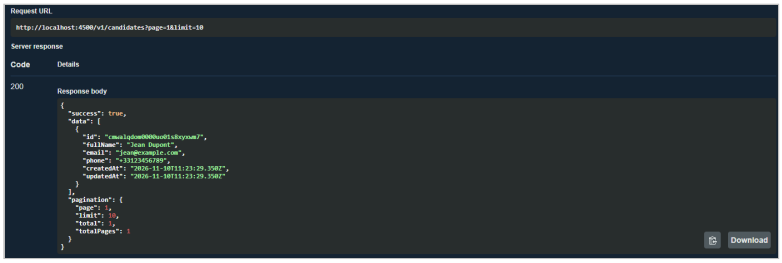


Figure 23 : candidat toujours présent après redémarrage